SIRE511 : LINUX AND BIOINFORMATICS DATA SKILLS

Bioinformatics Data Skill IV

Unix data tools

9th Week, 29/10/2024

Kwanrutai Mairiang, Ph.D

Outline

- Unix data tools
  - Awk
  - Bioawk
  - sed
- Working with sequence data
- Working with alignment data

Unix Data Tools

Remind: Commands for data manipulation

- grep, cut, join, uniq, sort
- Open “more_info_week9.docx” file

Text Processing with Awk

- Awk is a tool that lets user create small and powerful programs. These programs look for specific text patterns in each line of a document and perform action when they find a match.
- Basic syntax:

```
$ awk '{action}' your_file_name.txt
```

- Search for the text with specific pattern using regex

```
$ awk `/regex pattern/{action}'
your_file_name.txt
```

https://www.freecodecamp.org/news/the-linux-awk-command-linux-and-unix-usage-syntax-examples/
https://www.geeksforgeeks.org/awk-command-unixlinux-examples/

The examples of using awk command.

- To print all the contents of the file using awk

```
$ awk ‘{print $0}’ example.bed
```

- How to print specific columns using awk

```
$ awk '{ print $2 "\t" $3 }' example.bed
```

- Awk supports arithmetic with the standard operators +, -, *, /, % (remainder), and ^ (exponentiation).

```
$ awk ‘$3 - $2 > 18’ example.bed
```

The examples of using awk command (cont.).

- Counting and Printing Matched Pattern

```
$ awk '/chr1/{++count}; END {print "Count = "count}' example.bed
```

- Use logical operators &&, ||, and ! to combine conditions in AWK.

```
$ awk '$1 ~ /chr1/ && $3 - $2 > 10' example.bed
```

- Combine patterns and more complex actions than just print entire record.

```
$ awk '$1 ~ /chr2|chr3/ { print $0 "\t" $3 - $2 }' example.bed
```

https://www.digitalocean.com/community/tutorials/awk-command-linux-unix#counting-and-printing-matched-pattern

Comparison Operators in AWK

Operator	Detail
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
~	Matches a regular expression / Pattern
!~	Does not matches a regular expression / Pattern
&&	Logical AND
||	Logical OR
!	Logical NOT

The examples of using awk command. (cont.)

- Count number of each gene features in gtf file

```
awk '/Lypla1/ {feature[$3]++1}; END {for (k in
feature) print k "\t" feature[k]}'
Mus_musculus.GRCm38.75_chr1.gtf
```

- Generate a three-column BED file contain chromosome start end from gtf file.

```
$ awk '!/^#/ { print $1 "\t" $4-1 "\t" $5 }'
Mus_musculus.GRCm38.75_chr1.gtf
```

The examples of using awk command. (cont.)

- Two special patterns: BEGIN and END are optional in Awk.
- These patterns are typically used for executing code before the main processing begins (BEGIN) and after it ends (END).
- BEGIN is useful to initialize and set up variables, and END is useful to print data summaries at the end of file processing.
- Example: Calculate mean length from bed file

```
$ awk 'BEGIN{ s = 0 }; { s += ($3-$2) }; END{
print "mean: " s/NR };' example.bed
```

NR = line number NF = column number

More useful examples of using Awk

- https://phoenixnap.com/kb/awk-command-in-linux
- https://www.digitalocean.com/community/tutorials/awk-command-linux-unix
- https://www.geeksforgeeks.org/awk-command-unixlinux-examples/

Bioawk: An Awk for Biological Formats

- Bioawk is an extension of the AWK command that provides several features for biological data manipulation.
  - bed, sam, vcf, gtf/gff, fasta/fastq
- Install Bioawk

```
$ sudo apt update
$ apt install bioawk
```

- Let's check the supported input formats in Bioawk.

```
$ bioawk -c help
```

The example of using Bioawk command.

- Print the chromosome and gene size of protein-coding gene records in the GTF file.

```
$ bioawk -c gff '$3 ~ /gene/ && $2 ~ /protein_coding/ \
{print $seqname,$end-$start}'\
Mus_musculus.GRCm38.75_chr1.gtf
```

- Turn FASTQ file to FASTA file

```
$ bioawk -c fastx '{print ">"$name"\n"$seq}'
contam.fastq
```

- Counting the number of FASTQ/FASTA entries

```
$ bioawk -c fastx 'END{print NR}' contam.fastq
```

The example of using Bioawk command.

- Print the size of each chromosome from reference genome file

```
$ bioawk -c fastx '{print $name,length($seq)}' \
  Genome_ref.fa.gz
```

- Use Bioawk with plain tab-delimited files

```
$ bioawk -c hdr '$ind_A == $ind_B {print $id}’ \
genotypes.txt
```

hdr stands for "header”. Bioawk will automatically assign a name of each column based on the header row.

- For more useful example:
  - https://bioinformaticsworkbook.org/Appendix/Unix/bioawk-basics.html#gsc.tab=0

Stream Editing with Sed

- Sed command reads each line of text from input source (file, standard input, or pipeline) and edit a line at a time.
- After processing, sed command sends the modified text to the standard output.
- Sed has capabilities that overlap other Unix tools like grep and awk.

Syntax of sed command

- Print text
  - Print the whole file

    ```
    $ sed ` ` filename
    ```
  - Printing line

    ```
    $ sed –n ‘1,2p’ filename
    ```
- Deleting text
  - Use the same syntax as “print text” but replace ‘p’ by ‘d’

    ```
    $ sed ‘1,2d’ finename
    ```
- Substituting text
  - Replace the first occurrence of a match

    ```
    $ sed ’s/pattern/replacement/’
    ```
  - Replace all occurrence of strings that match the pattern

    ```
    $ sed ’s/pattern/replacement/g’
    ```
  - Match with the case-insensitive

    ```
    $ sed  ’s/pattern/replacement/i’
    ```

The examples of using sed command.

- Edit text from file

```
$ sed ‘s/chrom/chr/’ chroms.txt
```

- Edit text from stadard input

```
$ echo "chr1:28427874-28425431” | sed -E
's/^(chr[^:]+):([0-9]+)-([0-9]+)/\1\t\2\t\3/’
$ echo "chr1:28427874-28425431" | sed 's/[:-
]/\t/g’
$ echo "chr1:28427874-28425431" | sed
's/:/\t/' | sed 's/-/\t/’
$ echo "chr1:28427874-28425431" | sed -e
's/:/\t/' -e 's/-/\t/'
```

The examples of using sed command.

- Edit text from stadard input

```
$ echo "chr1:28427874-28425431” | sed -E
's/^(chr[^:]+):([0-9]+)-([0-9]+)/\1\t\2\t\3/’
```

CMD	Meaning
`sed -E`	-E enables extended regular expressions. This option allows for more advanced regular expressions without requiring you to escape certain special characters, such as (,), {}, +, and ?.
`[^:]+`	Any number of characters that is not a colon (:)
`([0-9]+)`	Captures a series of digits
\1, \2, and \3	This set represents the captured groups (chromosome, start, and end positions).
`sed -e`	-e allows chaining multiple sed expressions in a single command.

The examples of using sed command.

- Print text in specific line

```
$ sed -n '1,10p'
Mus_musculus.GRCm38.75_chr1.gtf
$ sed -n '20,50p'
Mus_musculus.GRCm38.75_chr1.gtf
```

- Delete text in specific line

```
$ sed '1,10d' Mus_musculus.GRCm38.75_chr1.gtf
$ sed '20,50d' Mus_musculus.GRCm38.75_chr1.gtf
```

- More sed example
  - https://www.geeksforgeeks.org/sed-command-in-linux-unix-with-examples/

Working with Sequence Data

Sequence format: Fasta, Fastq

FASTA Format

- The FASTA format is used to store any sort of sequence data (DNA, RNA, Protein). This format contains only sequence data and does not include quality scores.
- The FASTA file contain two parts.
  - The description line begins with a greater than symbol (>)
  - The sequence data begins on the next line after the description.

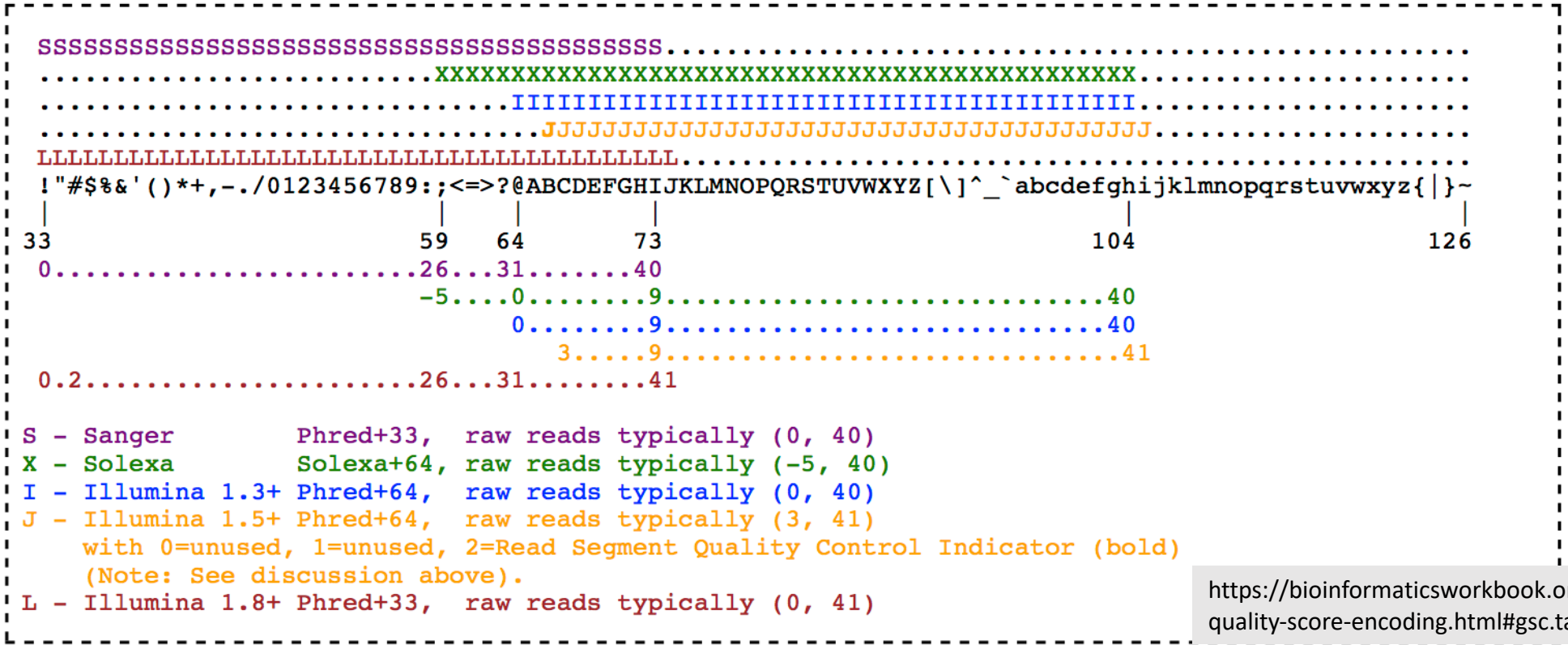
```
>ENSMUSG00000020122|ENSMUST00000138518
CCCTCCTATCATGCTGTCAGTGTATCTCTAAATAGCACTCTCAACCCCCGTGAACTTGGT
TATTAAAAACATGCCCAAAGTCTGGGAGCCAGGGCTGCAGGGAAATACCACAGCCTCAGT
TCATCAAAACAGTTCATTGCCCAAAATGTTCTCAGCTGCAGCTTTCATGAGGTAACTCCA
```

Sequence format: Fasta, Fastq

FASTQ Format

- The FASTQ format extends FASTA by including a numeric quality score to each base in sequence. FASTQ format is widely used to store high-throughput sequencing data.
- The FASTQ file contain four parts.
  - The description line begins with @ symbol
  - The sequence data begins on the next line after the description.
  - The line beginning with plus (+) symbol indicate the end of the sequence line.
  - The base quality data, each numeric base quality is encoded with ASCII characters.

FASTQ Format and Quality score



https://bioinformaticsworkbook.org/introduction/fastq-quality-score-encoding.html#gsc.tab=0

Illumina 1.8+

```
@DJB775P1:248:D0MDGACXX:7:1202:12362:49613
TGCTTACTCTGCGTTGATACCACTGCTTAGATCGGAAGAGCACACGTCTGAA
+
JJJJJIIJJJJJJHIHHHGHFFFFFFCEEEEEDBD?DDDDDDBDDDABDDCA
```

J = 41, I = 40, H = 39, G = 38, F = 37, E = 36, D = 35, C = 34, B = 33, A = 32, ? = 30

The example of working with sequence data?

- Count the number of sequences
  - Use grep command

```
$ grep -c “^>” egfr_flank.fasta
$ grep -c “^@” untreated1_chr4.fq
```

  - Does this grep command really work with FASTQ? Try bioawk…

```
$ bioawk -c fastx 'END {print NR}' untreated1_chr4.fq
```

- Get %GC

```
$ bioawk -c fastx '{ print $name, gc($seq) }’
sequence_file
```

- Get reverse complement

```
$ bioawk -c fastx '{ print ">"$name; print revcomp($seq)
}’ sequence_file
```

https://bioinformaticsworkbook.org/Appendix/Unix/bioawk-basics.html#gsc.tab=0

The example of working with sequence data?

- Print sequences with length greater than 100 bases

```
$ bioawk -c fastx 'length($seq)>100 { print ">"$name; print $seq }' sequence_file
```

- Add a prefix/suffix to the sequence defline

```
$ bioawk -c fastx '{ print ">PREFIX"$name; $seq }' sequence_file
```

```
$ bioawk -c fastx '{ print ">"$name"|SUFFIX"; $seq }' sequence_file
```

- Print name and sequence in tabular format

```
$ bioawk -t -c fastx '{ print $name, $seq }' sequence_file
```

https://bioinformaticsworkbook.org/Appendix/Unix/bioawk-basics.html#gsc.tab=0

The example of working with sequence data?

- Get the mean Phred quality score from fastq

```
$ bioawk -c fastx '{print ">"$name; print
meanqual($qual)}' input.fastq
```

- Filter reads based on average quality score
  - keep only reads with an average quality score above a threshold

```
$ bioawk -c fastx '{ if(meanqual($qual) > 30) { print
"@"$name"\n"$seq"\n+\n"$qual } }' input.fastq
```

https://bioinformaticsworkbook.org/Appendix/Unix/bioawk-basics.html#gsc.tab=0

Working with Alignment Data

SAM and BAM format

- SAM stands for Sequence Alignment/Mapping (SAM) format.
- A Binary Alignment Map (BAM) is a compressed binary version of a SAM file.
- The SAM and BAM formats are the standard formats for storing alignment between sequencing reads mapped to a reference.

http://samtools.github.io/hts-specs/SAMv1.pdf

SAM format

SAM Header

```
$ grep "^@" celegans.sam
@SQ    SN:I  LN:15072434
@SQ    SN:II LN:15279421
@SQ    SN:III      LN:13783801
@SQ    SN:IV LN:17493829        KEY:VALUE
@SQ    SN:MtDNA    LN:13794
@SQ    SN:V  LN:20924180
@SQ    SN:X  LN:17718942
@RG    ID:VB00023_L001   SM:celegans-01
@PG    ID:bwa      PN:bwa      VN:0.7.10-
r789  CL:bwa mem -R
@RG\tID:VB00023_L001\tSM:celegans-01
Caenorhabditis_elegans.WBcel235.dna.topl
evel.fa celegans-1.fq celegans-2.fq
```

Key	Description
@SQ	The information about the reference sequence. - SN = sequence name - LN = sequence length
@RG	Read group and sample metadata. - ID = read group identifier. ID is required and must be unique - SM = sample information - PL = sequencing platform
@PG	Metadata about alignment program - ID = unique ID value of program - VN = program version - CL = exact command line

More information about SAM format: http://samtools.github.io/hts-specs/

SAM format

Alignment Section

Col	Field	Type	Brief description
1	QNAME	String	Query template NAME
2	FLAG	Int	bitwise FLAG
3	RNAME	String	Reference sequence NAME11
4	POS	Int	1-based leftmost mapping POSition
5	MAPQ	Int	MAPping Quality
6	CIGAR	String	CIGAR string
7	RNEXT	String	Reference name of the mate/next read
8	PNEXT	Int	Position of the mate/next read
9	TLEN	Int	observed Template LENgth
10	SEQ	String	segment SEQuence
11	QUAL	String	ASCII of Phred-scaled base QUALity+33

SAM format

CIGAR string

Operation	Value	Description
M	0	Alignment match (note that this could be a sequence match or mismatch!)
I	1	Insertion (to reference)
D	2	Deletion (from reference)
N	3	Skipped region (from reference)
S	4	Soft-clipped region (soft-clipped regions are present in sequence in SEQ field)
H	5	Hard-clipped region (not in sequence in SEQ field)
P	6	Padding (see section 3.1 of the SAM format specification for detail)
=	7	Sequence match

Working with SAM/BAM file

- An example of an alignment command to generate a SAM file.
  - `$ bwa mem -R'@RG\tID:readgroup_id\tSM:sample_id' ref.fa in.fq`
- The standard program use for working with SAM/BAM file is “samtools”
- Install samtools
  - Install by apt install:

    `$ sudo apt install samtools`
  - Install by Conda

    `$ conda install -c bioconda samtools`
  - Samtools in docker image
    - Pull Docker

      `$ docker pull staphb/samtools`
    - Run docker image

      `$ docker run -it -v /your/local/path:/data -w /data staphb/samtools /bin/bash`

Use samtools for working with SAM/BAM

- List all commands

```
$ samtools
$ samtools command
```

- Convert SAM to BAM

```
$ samtools view -b celegans.sam > celegans.bam
```

- Convert BAM to SAM

```
$ samtools view -h celegans.bam >
celegans_copy.sam
```

- Sort alignment by their alignment position

```
$ samtools sort -m 1G -@ 2 celegans.bam >
celegans.sorted.bam
```

Extracting alignments from a region with samtools view

- First, index bam file

```
$ samtools index NA12891_CEU_sample.bam
```

- Extract alignment result from a specific region

```
$ samtools view NA12891_CEU_sample.bam
1:215906469-215906652
```

- Extract alignment result from specific regions in bed file

```
$ samtools view -L USH2A_exons.bed
NA12891_CEU_sample.bam
```

Filtering alignments using FLAGs

```
$ samtools flags
```

Flag (hexidecimal)	Flag	Flag	Meaning
0x1	1	PAIRED	Paired-end sequence (or multiple-segment, as in strobe sequencing)
0x2	2	PROPER_PAIR	Aligned in proper pair (according to the aligner)
0x4	4	UNMAP	Unmapped
0x8	8	MUNMAP	Mate pair unmapped (or next segment, if mulitple-segment)
0x10	16	REVERSE	Sequence is reverse complemented
0x20	32	MREVERSE	Sequence of mate pair is reversed
0x40	64	READ1	The first read in the pair (or segment, if multiple-segment)

Filtering alignments using FLAGs with samtools view

- There are two options related to filter alignment based on flags

```
-f : only outputs reads with the specified
flag(s)
-F : only outputs reads without the specified
flag(s)
```

- Filter only unmapped read

```
$ samtools flags unmap  # Check decimal flag
$ samtools view -f 4 NA12891_CEU_sample.bam
```

- Filter READ1 that mapped in PROPER_PAIR

```
$ samtools flags READ1,PROPER_PAIR
$ samtools view -f 66 NA12891_CEU_sample.bam
```

Filtering alignments using FLAGs with samtools view

- Filter only PROPER_PAIR

```
$samtools view -f 2 NA12891_CEU_sample.bam
```

- Filter out all UNMAP reads

```
$ samtools view -F 4 NA12891_CEU_sample.bam
```